

Relatório de Teste Funcional com Playwright

Testes de caixa-preta das interfaces de cálculo de frete e de cadastro de senha

Pedro Massaki Carnelossi

RA 24036481-2

André Filipe Caetano Mulati

RA 24011149-2

Este relatório descreve os testes funcionais escritos para as duas interfaces pedidas na atividade: cálculo de frete e cadastro de senha. A aplicação didática e os specs de exemplo (login e idade) são o material fornecido pelo professor e foram mantidos sem qualquer alteração. O trabalho da dupla está em `tests/frete.spec.ts`, `tests/senha.spec.ts` e no apoio em `tests/support/`.

1. Objetivo e escopo

O objetivo foi verificar, pela interface e sem consultar o código-fonte das telas, se cada regra anunciada na própria página é cumprida. Nenhum teste inspeciona variáveis internas da aplicação: a conferência é feita pelo texto exibido ao usuário e pelo papel de acessibilidade do parágrafo de resposta — `status` quando a operação é aceita e `alert` quando é recusada. Esse par de papéis funciona como o oráculo de todos os cenários.

Ficaram fora do escopo desempenho, aparência das telas e compatibilidade entre navegadores. A configuração do projeto mantém Firefox e WebKit disponíveis, mas a execução registrada aqui é a do Chromium, conforme o script `npm test`.

2. Técnicas aplicadas

- **Particionamento em classes de equivalência** — cada campo foi dividido em uma classe válida e nas classes inválidas que a regra distingue. No frete, por exemplo, o CEP tem a classe válida de oito dígitos e as classes inválidas de comprimento, de caractere e de formatação.
- **Análise de valor-limite** — em toda fronteira foram testados o próprio limite e os dois vizinhos imediatos: R\$ 199,99 / R\$ 200,00 / R\$ 200,01 no frete grátis, e senhas de 7, 8, 19, 20 e 21 caracteres nos limites de tamanho.
- **Teste de sequência** — cenários que reenviam o formulário depois de um erro, para verificar que a tela se recupera e que a ordem das verificações é respeitada.

3. Organização do projeto

A aplicação sob teste é a fornecida na atividade e não foi modificada: `server.js` roteia as telas e `public/` guarda as páginas, os scripts e a folha de estilo. Os specs de exemplo `tests/login.spec.ts` e `tests/idade.spec.ts` também permanecem como vieram, servindo de referência de estilo e de verificação de regressão.

Os arquivos escritos pela dupla são `tests/frete.spec.ts` e `tests/senha.spec.ts`. Os seletores e as ações não aparecem dentro deles: ficam em objetos de tela (*Page Objects*) em `tests/support/telas.ts`, que expõem métodos como `calcularFrete(cep, valor)` e `deveRecusarCom(mensagem)`. Assim os specs descrevem apenas o cenário e o resultado esperado, e uma mudança de rótulo na interface é corrigida em um único lugar. As senhas de tamanho-limite são geradas por `senhaComTamanho(n)`, em

tests/support/dados.ts , o que evita contar caracteres à mão e deixa explícito qual valor-limite cada cenário exercita.

4. Cenários por funcionalidade

Tabela 1 — Distribuição dos cenários. As duas primeiras linhas são a entrega da dupla.

Tela	Grupos de cenários	Exemplos de valores exercitados	Execuções
Cálculo de frete (entrega)	tarifa por região; frete grátis; CEP inválido; valor inválido; combinações entre os dois campos; correção após erro	CEP 80000000 / 01000000 / 89999999 / 79999999; valores 199,99 / 200 / 200,01; CEP com 7 e 9 dígitos, com hífen e com espaço; valores 0, -0.01, 100.001, 1e2, 1.000,00 e Infinity	30
Cadastro de senha (entrega)	tamanho; composição obrigatória; espaços em branco; conferência da confirmação; ordem das verificações	senhas de 7, 8, 9, 19, 20 e 21 caracteres; sem maiúscula, sem minúscula e sem algarismo; espaço no início, no meio e no fim; tabulação; confirmação divergente e vazia	23
Acesso ao sistema (exemplo do professor)	credencial válida e credencial inválida	ana@exemplo.com com a senha correta e com senha incorreta	2
Validação de idade (exemplo do professor)	classes de equivalência e valores-limite da faixa de 18 a 65	17, 18, 19, 64, 65, 66, "dezoito", 18.5 e campo vazio	9

Pontos que mereceram cenário próprio

- **O frete grátis prevalece sobre a região.** Um pedido de R\$ 200,00 é isento tanto no CEP iniciado por 8 quanto nos demais, e os dois casos foram verificados separadamente.
- **Espaços em volta são descartados, espaços internos não.** O CEP " 80000000 " com o valor " 100,50 " continua válido; já "8000 000" é recusado.
- **O formato da senha é verificado antes da confirmação.** Com senha curta e confirmação diferente, a tela precisa acusar "Senha fora do padrão", e não a divergência. Os dois erros foram combinados de propósito para fixar essa ordem.
- **Limpeza do formulário.** Depois de um cadastro aceito os dois campos de senha ficam vazios; depois de uma recusa, permanecem preenchidos.
- **Entradas que o JavaScript aceitaria.** 1e2 e Infinity são números válidos para a linguagem, mas não para a regra da tela; ambos foram testados.

5. Resultado da execução

64 TESTES EXECUTADOS	64 APROVADOS	0 FALHAS	0 IGNORADOS
-------------------------	-----------------	-------------	----------------

Dos 64 testes, **53 são a entrega da dupla** — 30 de frete e 23 de senha — e 11 são os exemplos que acompanham o projeto. A suíte roda em cerca de 13 segundos com execução paralela. Ambiente da execução registrada: Windows 11, Node.js v22.15.0, Playwright 1.62.1 e Chromium 151 (build 1234). Nenhum teste precisou de repetição, e a aplicação é iniciada e encerrada pelo próprio Playwright.

Tabela 2 — Execuções por arquivo, no projeto chromium.

Arquivo de teste	Funcionalidade	Execuções	Resultado
------------------	----------------	-----------	-----------

Arquivo de teste	Funcionalidade	Execuções	Resultado
tests/frete.spec.ts	Cálculo de frete — entrega	30	aprovado
tests/senha.spec.ts	Cadastro de senha — entrega	23	aprovado
tests/login.spec.ts	Acesso — exemplo do professor	2	aprovado
tests/idade.spec.ts	Idade — exemplo do professor	9	aprovado
Total		64	sem falhas

6. Como executar

1. Abrir o terminal na pasta da atividade.
2. `npm install` — instala as dependências.
3. `npm run browsers` — baixa o Chromium usado pelo Playwright.
4. `npm test` — executa a suíte no Chromium; `npm run report` abre o relatório em HTML da última execução.

Para acompanhar o navegador durante a execução, `npm run test:headed`; para investigar uma falha passo a passo, `npm run test:ui`.

7. Considerações finais

Os 53 cenários entregues cobrem as classes válidas, as classes inválidas e os valores-limite descritos nas próprias telas de frete e de senha, e todos responderam conforme o esperado. Vale registrar o que a suíte *não* demonstra: por se tratar de teste funcional, passar em todos os cenários não prova ausência de defeito, apenas que as regras verificadas se comportam corretamente para as entradas escolhidas. Regras não anunciadas na interface — por exemplo, a existência real de um CEP — continuam fora do alcance destes testes.